

Classification Algorithm

XGBoost

Genetic Algorithm

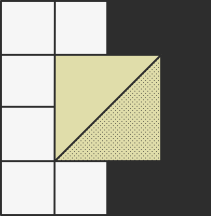
Group 7

Lam Yoke Yu A23CS0233

Tan Yi Ya A23CS0187

Tan Zhi Ming A23CS0189





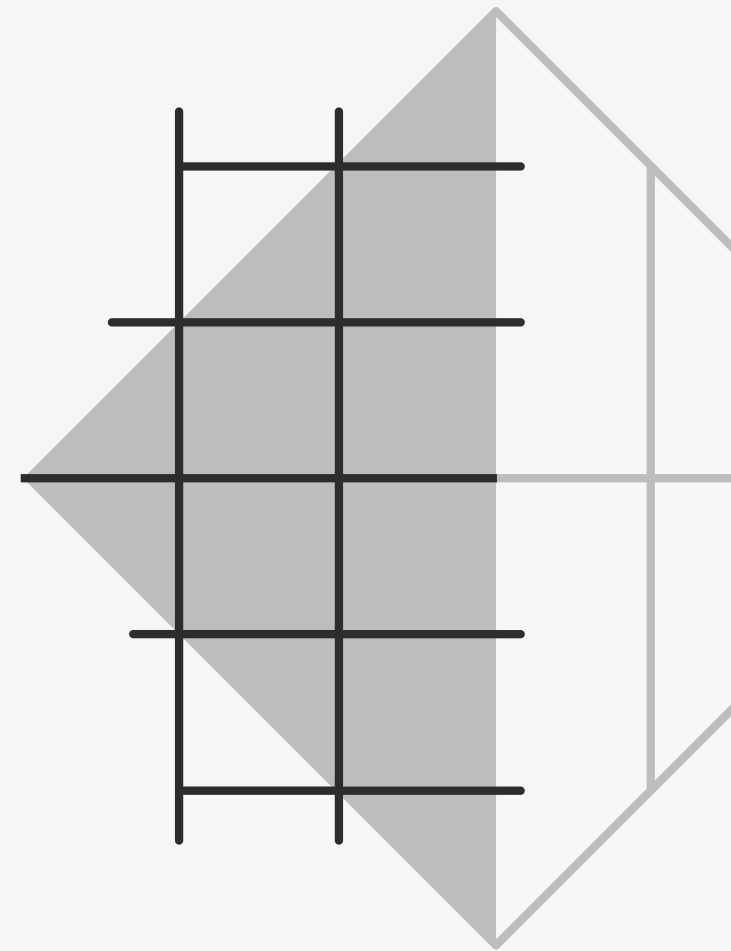
XGBoost

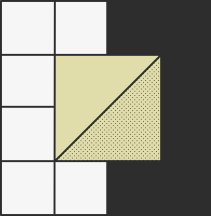
- short for **eXtreme Gradient Boosting**
- an optimized distributed **gradient boosting library**
- a type of **ensemble learning** method
- provides a **parallel tree boosting** (also known as GBDT)
- uses decision trees as its base learners combining them sequentially to improve the model's performance

Source

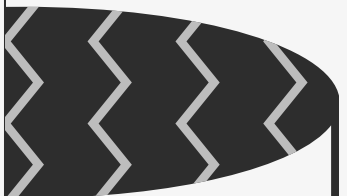
https://xgboost.readthedocs.io/en/release_3.0.0/index.html

<https://www.geeksforgeeks.org/xgboost/>





How XGBoost Works



1. Start with a base learner

The **first model decision tree** is trained on the data

2. Calculate the errors

The **errors** between the predicted and actual values are calculated

3. Train the next tree

The next tree is trained on the errors of the previous tree and **attempts to correct the errors** made by the first tree

4. Repeat the process

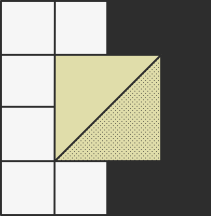
This process continues with each new tree trying to correct the errors of the previous trees (boosting) until a stopping criterion is met

5. Combine the predictions

The errors between the predicted and actual values are calculated

Source

<https://www.geeksforgeeks.org/xgboost/>



Advantages and Disadvantages of XGBoost

Advantages	Disadvantages
highly scalable and efficient	less suitable for resource-constrained systems
suitable for big data applications and real-time predictions	sensitive to noisy data or outliers
allow users to fine-tune the model according to their specific needs	prone to overfitting on small datasets
feature importance analysis	challenging to interpret

Source: <https://www.geeksforgeeks.org/xgboost/>

Suitable types of data for XGBoost

Data type	Notes
Numerical	XGBoost naturally handles numerical data without the need for scaling or normalization, as tree-based models are not sensitive to the scale of features.
Categorical	XGBoost can handle categorical data effectively when encoded appropriately, allowing the model to learn from categorical relationships.
Binary	Binary features (0/1) are just numerical and handled well
Ordinal	Treat as numerical or categorical depending on context

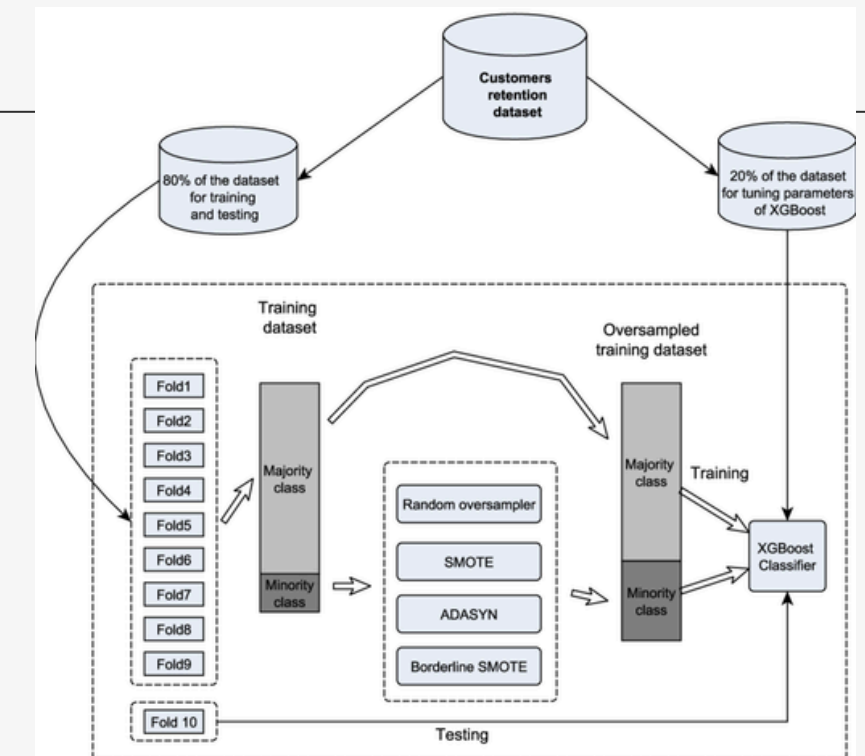
Source: <https://medium.com/low-code-for-advanced-data-science/xgboost-explained-a-beginners-guide-095464ad418f>
https://xgboost.readthedocs.io/en/release_3.0.0/
<https://xgboosting.com/>

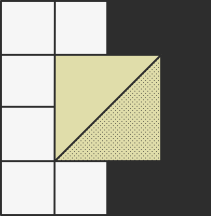
Example of XGBoost

Banks aim to predict which customers are likely to churn.

XGBoost constructs an ensemble of decision trees in a sequential manner. Each tree is trained to **correct the errors made by the previous trees** by minimizing a specified loss function. Through this gradient boosting process, XGBoost efficiently **captures complex, non-linear relationships and interactions within the features**.

By learning these patterns from historical customer data, XGBoost outputs a predictive model that **estimates the probability of each customer churning**. This allows the bank to proactively **identify high-risk customers and design targeted retention strategies**.





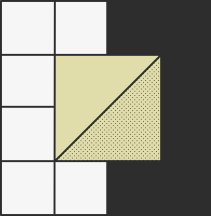
How Genetic Algorithm Works

Overview

- Adaptive heuristic search algorithms
- Simulate the process of natural selection and genetics – only species that can adapt to environmental change survives and go to the next generation

Key Terms

- **Genome:** Solution represented by binary encoding
- **Generation:** Set of all current solution at a given point during the algorithm
- **Fitness Function:** Rates how good a solution is (higher = better)
- **Selection Function:** Choosing the fittest individuals for reproduction
- **Crossover Function:** Mixing two parent chromosomes to create offspring
- **Mutation Function:** Randomly altering parts of a chromosome to introduce variation



How Genetic Algorithm Works

1. Initialize Population

- Generate random solutions
- Size: e.g., 20–100 individuals (Solution)

2. Evaluate Fitness

- Use the fitness function to score each individual

3. Select Parents

- Better solutions are more likely to be selected to produce offspring

4. Crossover

- Combine parts of two parents to create new offspring
- Common types: single-point, two-point, uniform crossover

5. Mutation

- Randomly change genes with low probability (e.g., 1%)

6. Form New Generation

- Replace old population with new individuals (while keeping the best with elitism)

7. Repeat

- Run the loop for many generations or until solution converges

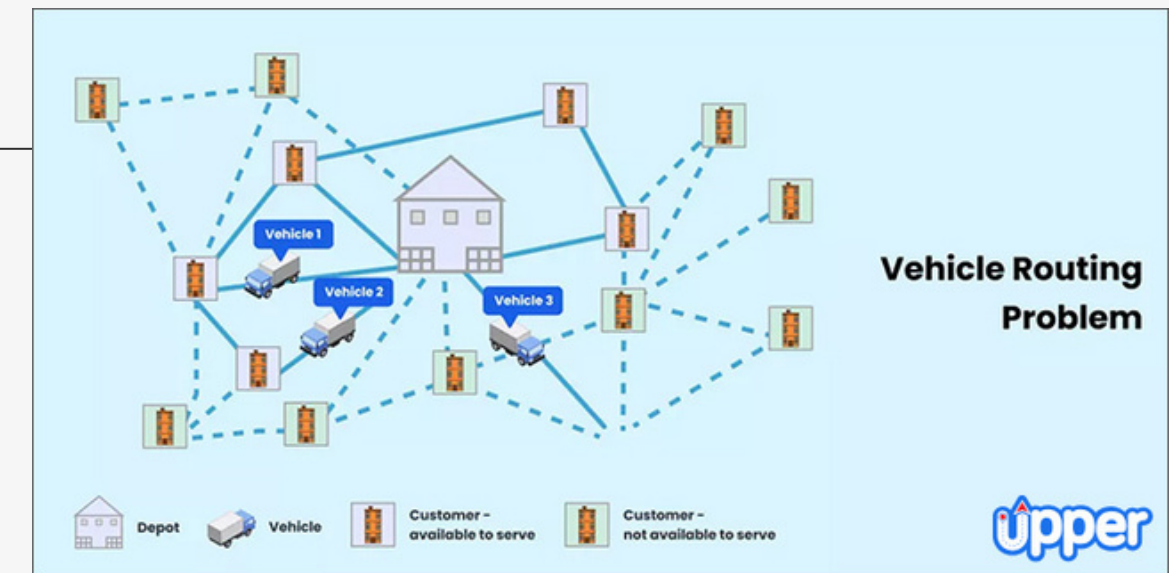
- Genetic representation of a solution
- Function to generate new solutions
- Fitness Function
- Selection Function
- Crossover Function
- Mutation Function

Suitable types of data for Genetic Algorithm

Data type	Notes
Binary	Simple, natural for discrete problems; easy to apply genetic operations.
Real Numbers	Enables precise control over continuous parameters; supports smooth mutation/crossover.
Integers	Matches discrete problem domains; preserves solution validity.
Categorical	Allows symbolic data handling; encoded into integers or binaries for GA manipulation.

Source: <https://help.imsl.com/c/2016/html/cnlstat/index.html>

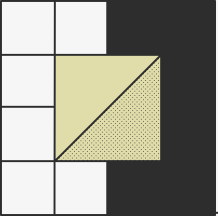
Example of Genetic Algorithms



Genetic algorithms work by copying how nature chooses the best traits to survive and improve over time. Imagine we have many possible routes for vehicles to take, like different ways to deliver packages.

- **Selection** : The algorithm picks the better routes — those that save more time, use less fuel, or cover shorter distances.
- **Crossover and Mutation** : It then mixes parts of these good routes together and makes small random changes to create new routes, hoping some will be even better.
- **Repeat** : This process happens again and again, each time keeping the best routes and trying to improve them.

Example: In the **Vehicle Routing Problem (VRP)**, where we need to figure out the best paths for many vehicles to deliver goods, genetic algorithms help find efficient routes that save time, fuel, and distance.



Thank you
for listening

